

Spring Framework



JDBC Template

JDBC Template로 축약

BookDao 기존 코드

전체 조회 getAllBook

```
public List<BookVo> getAllBook() {
    String sql = "SELECT * FROM book ORDER BY b_no";
    List<BookVo> books = new ArrayList<>();

    try {
        conn = dataSource.getConnection();    // ① 연결 열기
        pstmt = conn.prepareStatement(sql);
        rs = pstmt.executeQuery();
        while (rs.next()) {                   // ② 반복
            BookVo book = new BookVo();       // ③ 한 행 변환
            book.setB_no(rs.getInt("b_no"));
            book.setB_name(rs.getString("b_name"));
            book.setB_author(rs.getString("b_author"));
            /* ...컬럼 수만큼 계속... */
            books.add(book);
        }
    } catch (Exception e) {                   // ④ 예외 처리
        e.printStackTrace();
    }
    return books;    // 연결 닫기도 원래는 직접...
}
```

BookDao 축약 코드

BookDao — 같은 기능이 4줄로

```
public List<BookVo> getAllBook() {
    String sql = "SELECT * FROM book ORDER BY b_no";
    return jdbcTemplate.query(sql, new BookRowMapper());
}
```

왼쪽 코드의 변화

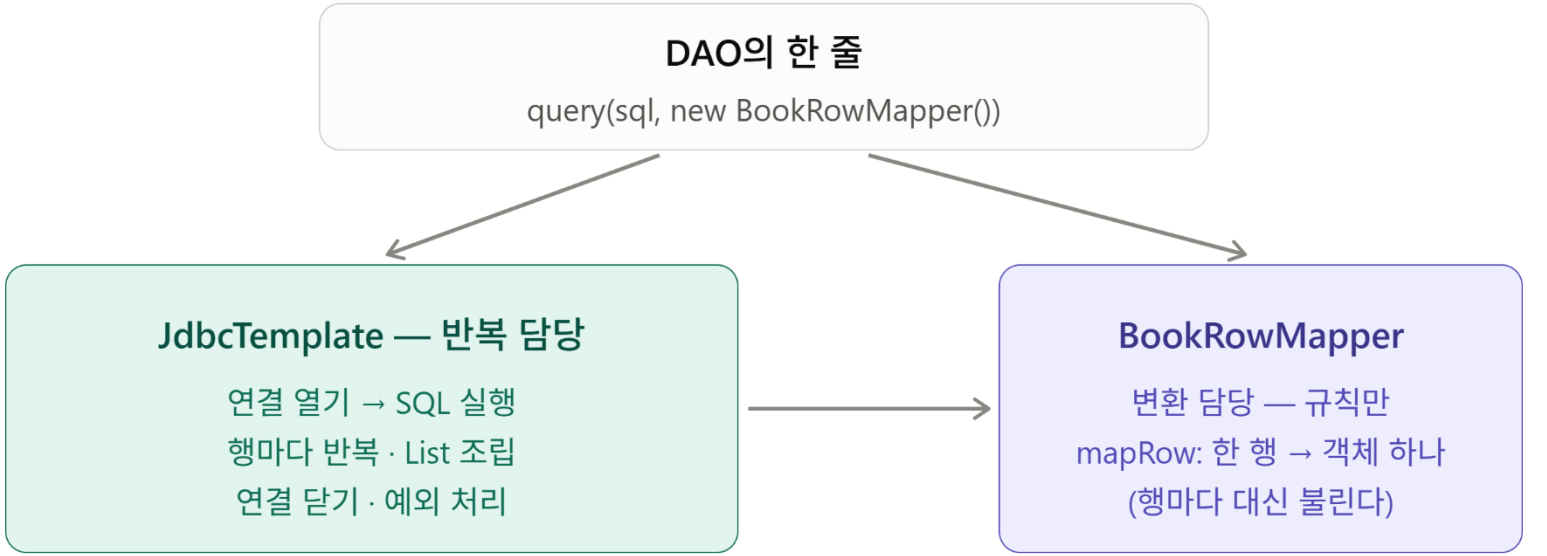
① 연결 열기·닫기 / ④ try-catch ➡ JdbcTemplate이 내부에서 대신 처리

② while(rs.next()) 반복 ➡ query()가 행 수만큼 알아서 반복

③ 한 행 → BookVo 변환 코드 ➡ **유일하게 살아남아 mapRow()로 이사**

"몇 줄이든 반복해 List로 모으는 일"은 JdbcTemplate의 몫,
"한 줄을 객체로 바꾸는 규칙"만 우리가 RowMapper에 정의.

JDBC Template로 축약



getAllBook() 한 번의 내부 — 결과가 3행이라면:



행 수만큼 mapRow가 호출된다 — 3행이면 3번, 0행이면 0번(빈 List)

DBconfig Bean 추가

```java
@Bean
public JdbcTemplate jdbcTemplate(DataSource dataSource) {
    return new JdbcTemplate(dataSource);
}
```

BookRowMapper – New Class

```
public class BookRowMapper implements RowMapper<BookVo> {
    @Override
    public BookVo mapRow(ResultSet rs, int rowNum) throws SQLException {
        BookVo book = new BookVo();
        book.setB_no(rs.getInt("b_no"));
        book.setB_name(rs.getString("b_name"));
        book.setB_author(rs.getString("b_author"));
        book.setB_publisher(rs.getString("b_publisher"));
        book.setB_publish_year(rs.getString("b_publish_year"));
        book.setB_isbn(rs.getString("b_isbn"));
        book.setB_call_number(rs.getString("b_call_number"));
        book.setB_rental_able(rs.getInt("b_rental_able"));
        book.setB_img(rs.getString("b_img"));

        return book;
    }
}
```

BookDAO

```java
@Repository
public class BookDao {

    private JdbcTemplate jdbcTemplate;

    @Autowired
    public BookDao(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }
}
```

- INSERT, UPDATE, DELETE – jdbcTemplate.update(sql, 값..)
- SELECT(List) – jdbcTemplate.query(sql, new RowMapper()) - List
- SELECT – jdbcTemplate.queryForObject(sql, new RowMapper(), 값) - Object

조회(Select) - List

조회(Select) - List

■ BookDao에 Select Query 추가

```
public List<BookVo> getAllBook() {
    String sql = "SELECT * FROM book ORDER BY b_no";
    return jdbcTemplate.query(sql, new BookRowMapper());
}
```

■ BookController에 조회 쿼리 적용될 URL 연결

```
@RequestMapping("/booklist")
public String bookList(Model model) {
    List<BookVo> books = bookDao.getAllBook();
    model.addAttribute("books", books);
    return "book/book_list";
}
```

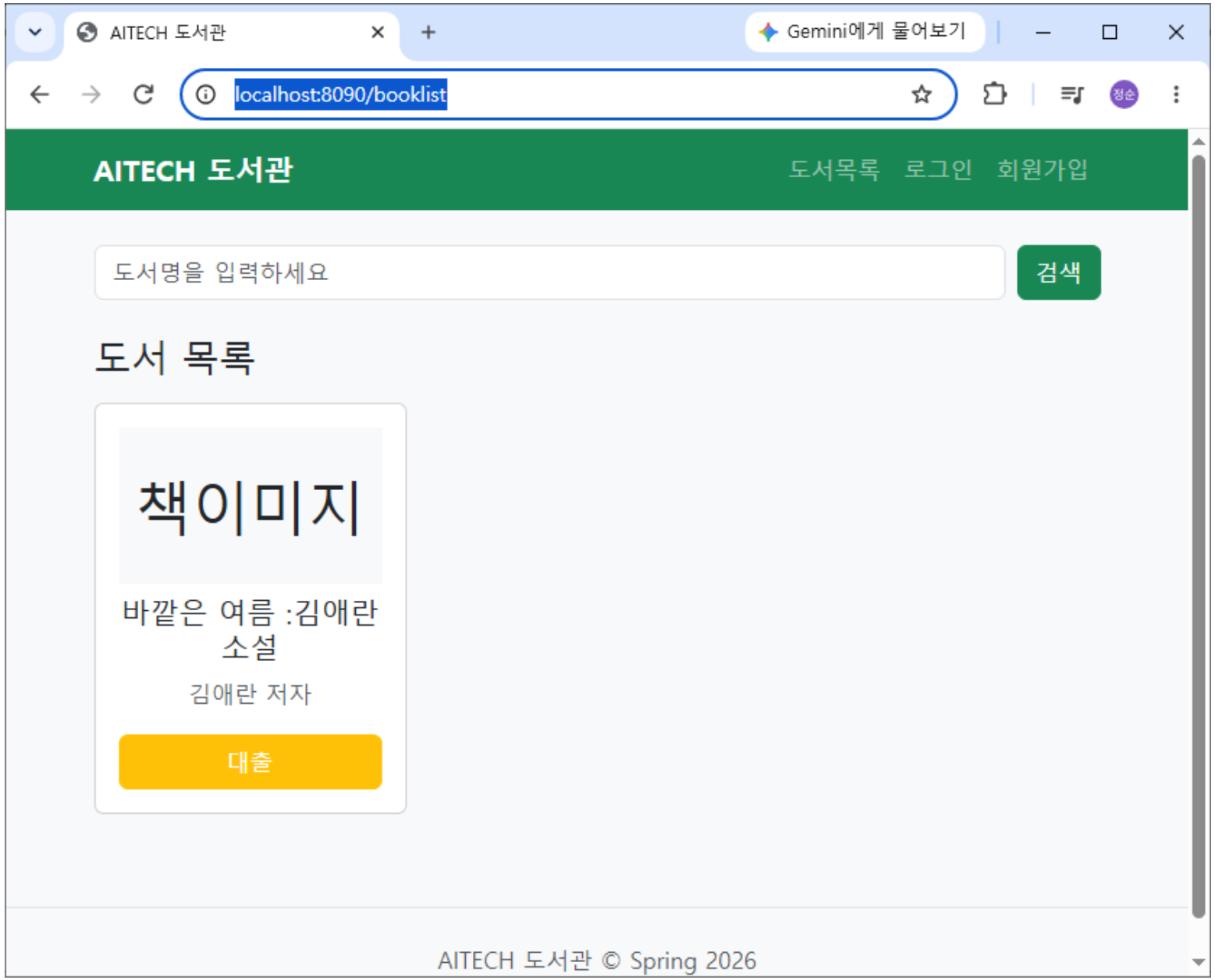
조회(Select) - List

book_list 내용 출력 수정

- addAttribut("books") = items="${books}"
- var="book" = ${book.~} / ex) ${book.b_name}, ${book.b_isbn} 등
- ${book.b_name} = getter(getB_name())

```
<c:forEach var="book" items="${books}">
    <div class="col-md-4 mb-3">
        <div class="card">
            <div class="card-body text-center">
                <div class="bg-light py-4 mb-2" style="font-size:40px">책이미지</div>
                <h5><a href="bookdetail?b_no=${book.b_no}"
                    class="text-decoration-none text-dark">${book.b_name}</a></h5>
                <p class="text-muted">${book.b_author} 저자</p>
                <div class="d-grid">
                    <button class="btn btn-warning text-white">대출</button>
                </div>
            </div>
        </div>
    </div>
</c:forEach>
```

도서목록 참고화면


AITECH 도서관
Gemini에게 물어보기
localhost:8090/booklist
AITECH 도서관
도서목록 로그인 회원가입
도서명을 입력하세요
검색
도서 목록
책이미지
바깥은 여름 :김애란 소설
김애란 저자
대출
AITECH 도서관 © Spring 2026

등록(Insert)

등록(Insert)

■ BookDao에 Insert Query 추가

```
public void insertBook(BookVo bookVo) {
    String sql = "INSERT INTO book(b_name, b_author, b_publisher, b_publish_year, " +
                 "b_isbn, b_call_number, b_rental_able, b_img, " +
                 "b_reg_date, b_mod_date) VALUES(?, ?, ?, ?, ?, ?, ?, ?, now(), now())";
    jdbcTemplate.update(sql, bookVo.getB_name(), bookVo.getB_author(), bookVo.getB_publisher(),
            bookVo.getB_publish_year(), bookVo.getB_isbn(), bookVo.getB_call_number(),
            bookVo.getB_rental_able(), bookVo.getB_img());
}
```

■ BookController에 조회 쿼리 적용될 URL 연결

```
@RequestMapping("/registerbook")
public String registerbook() {
    return "admin/register_book";
}

@RequestMapping("/registerbookconfirm")
public String registerbookconfirm(BookVo bookVo) {
    bookDao.insertBook(bookVo);
    return "admin/register_book";
}
```

등록(Insert)

book_list 내용 출력 수정

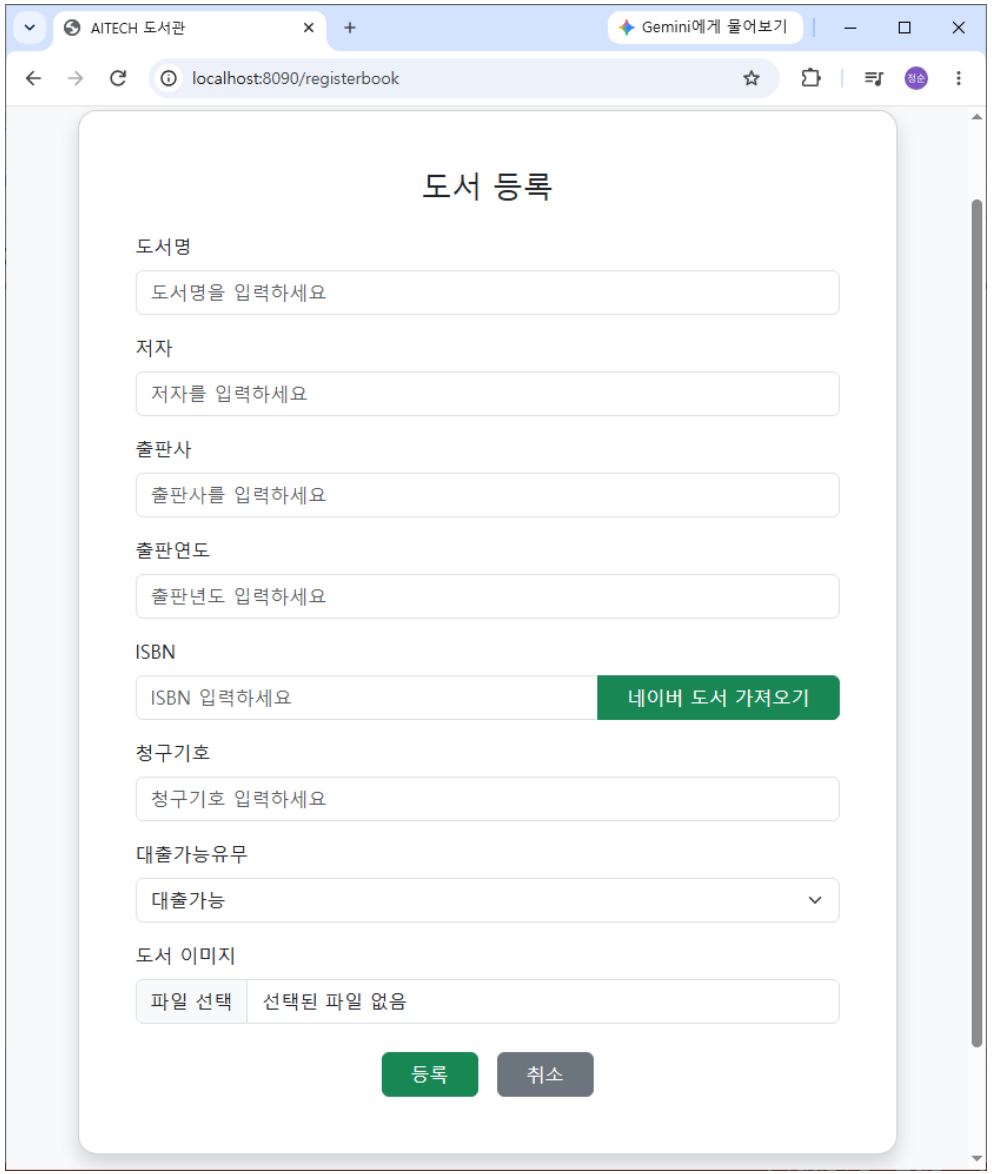
- <form> 태그 전송은 <button type="submit">
- 단순 페이지 이동은 <a href>

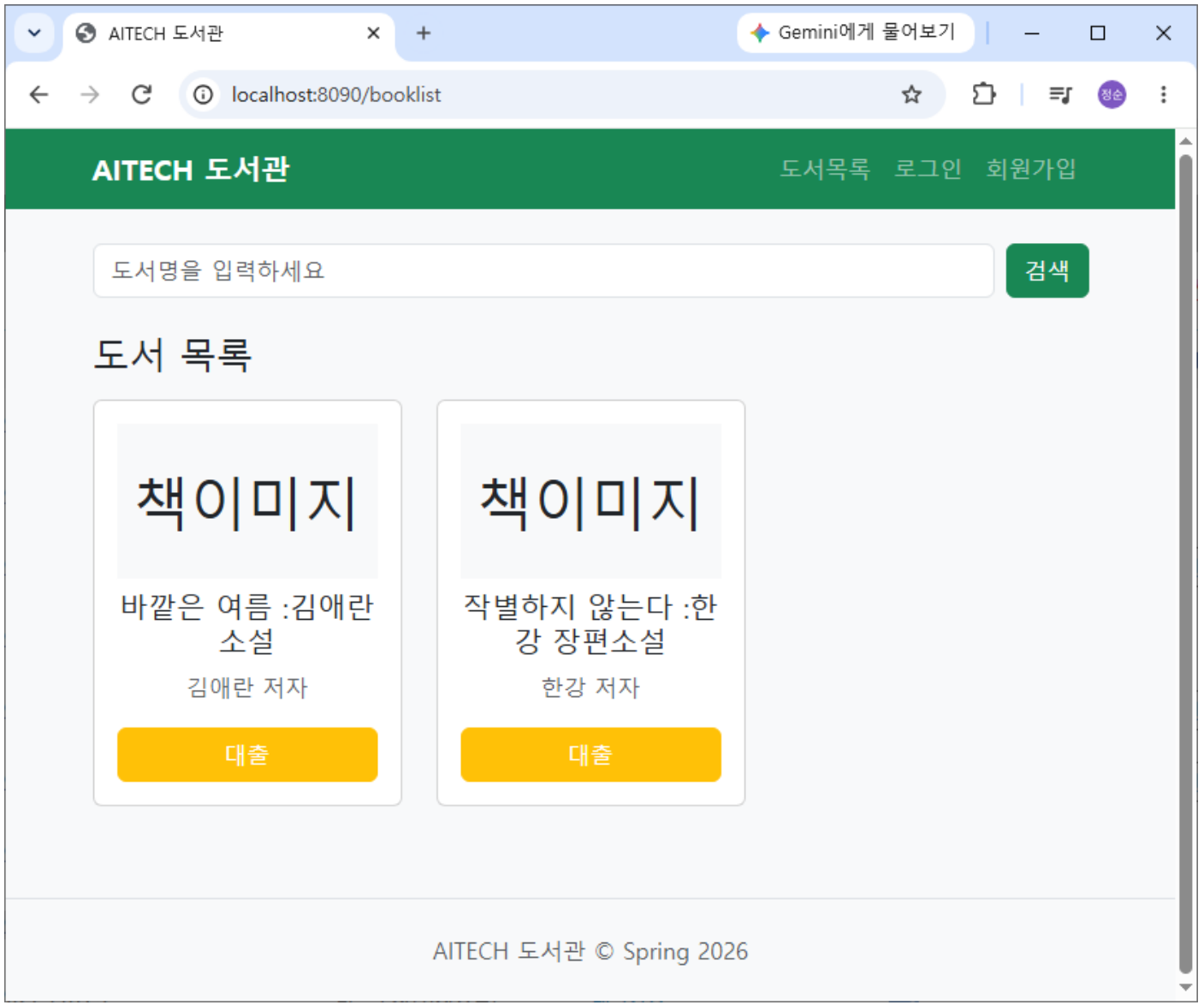
```
<form method="post" action="registerbookconfirm">

    <%-- 기존 input 그대로 (name = VO 필드와 동일하게) --%>

    <div class="d-flex justify-content-center gap-3">
        <button type="submit" class="btn btn-success px-4">등록</button>
        <a href="managementbook" class="btn btn-secondary px-4">취소</a>
    </div>
</form>
```

등록 참고화면


AITECH 도서관
Gemini에게 물어보기
localhost:8090/registerbook
도서 등록
도서명
도서명을 입력하세요
저자
저자를 입력하세요
출판사
출판사를 입력하세요
출판연도
출판년도 입력하세요
ISBN
ISBN 입력하세요
네이버 도서 가져오기
청구기호
청구기호 입력하세요
대출가능유무
대출가능
도서 이미지
파일 선택
선택된 파일 없음
등록
취소


AITECH 도서관
Gemini에게 물어보기
localhost:8090/booklist
AITECH 도서관
도서목록
로그인
회원가입
도서명을 입력하세요
검색
도서 목록
책이미지
바깥은 여름 :김애란 소설
김애란 저자
대출
책이미지
작별하지 않는다 :한강 장편소설
한강 저자
대출
AITECH 도서관 © Spring 2026

조회(Select) - Object

조회(Select) - Object

■ BookDao에 Select Query 추가

```
public BookVo getDetailBook(int b_no) {
    String sql = "SELECT * FROM book WHERE b_no = ?";
    return jdbcTemplate.queryForObject(sql, new BookRowMapper(), b_no) ;
}
```

■ BookController에 조회 쿼리 적용될 URL 연결

```
@RequestMapping("/bookdetail")
public String bookDetail(@RequestParam("b_no") int b_no, Model model) {
    model.addAttribute("book", bookDao.getDetailBook(b_no));
    return "book/book_detail";
}
```

조회(Select) - Object

■ book_detail 내용 출력 수정

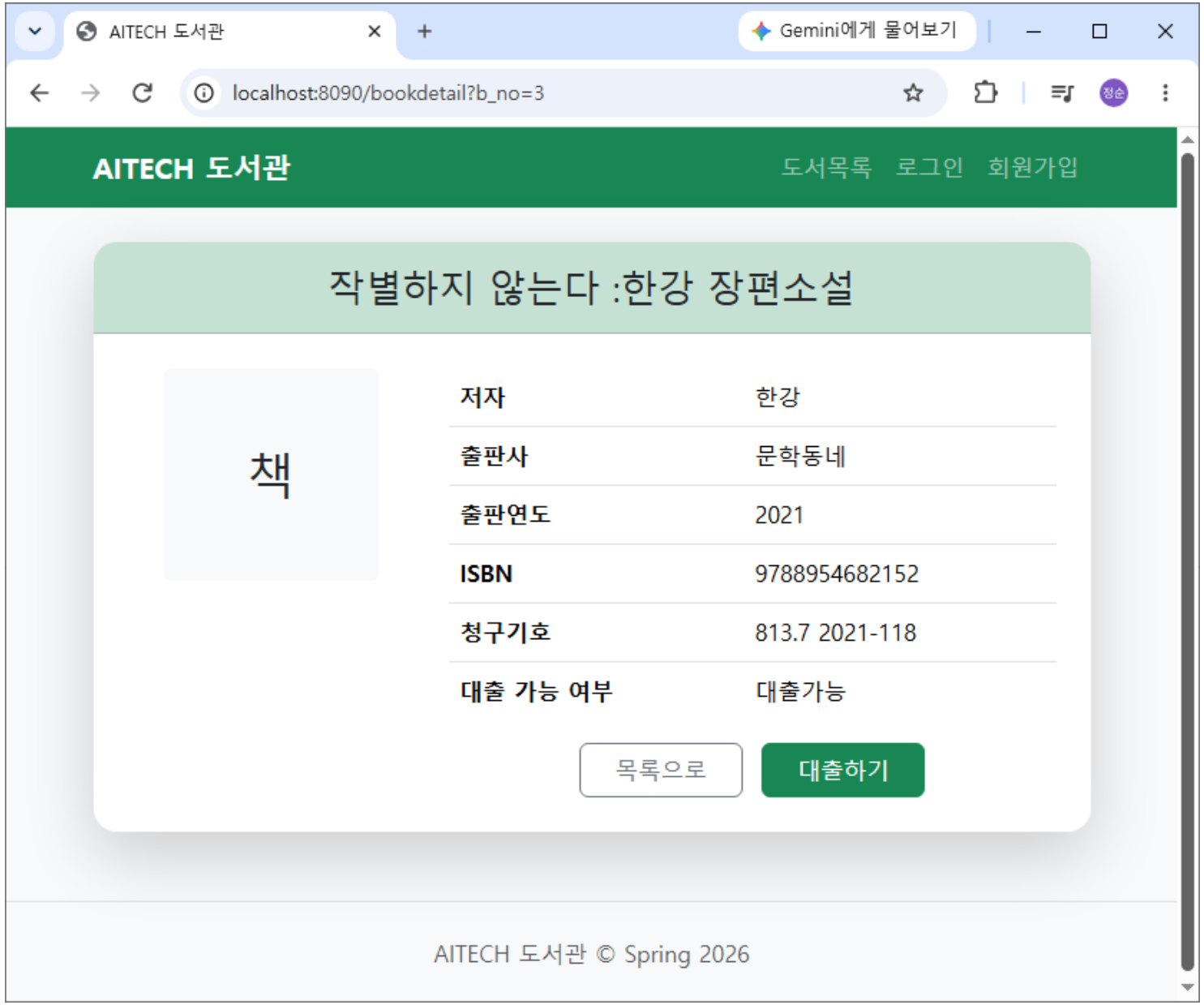
```
<div class="card-header bg-success bg-opacity-25 text-center py-3 rounded-top-4">
    <h3 class="mb-0">${book.b_name}</h3>
</div>
<div class="card-body p-4">
    <div class="row">
        <div class="col-md-4 text-center mb-3 mb-md-0">
            <div class="bg-light rounded py-5 fs-1 w-75 mx-auto">책</div>
        </div>
        <div class="col-md-8 d-flex flex-column">
            <table class="table table-borderless flex-grow-1">
                <tbody>
                    <tr class="border-bottom">
                        <th scope="row">저자</th>
                        <td>${book.b_author}</td>
                    </tr>
                    <%-- Vo에 맞춰 적용 -->
```

조회(Select) - Object

■ book_detail 내용 출력 수정

```
                    <tr>
                        <th scope="row">대출 가능 여부</th>
                        <td>
                            <c:choose>
                                <c:when test="${book.b_rental_able == 1}">대출가능</c:when>
                                <c:otherwise>대출불가</c:otherwise>
                            </c:choose>
                        </td>
                    </tr>
                </tbody>
            </table>
            <div class="mt-auto text-center">
                <a href="booklist" class="btn btn-outline-secondary px-4 me-2">목록으로</a>
                <a href="#" class="btn btn-success px-4">대출하기</a>
            </div>
        </div>
    </div>
</div>
```

상세정보 참고화면


AITECH 도서관
Gemini에게 물어보기
localhost:8090/bookdetail?b_no=3
AITECH 도서관
도서목록 로그인 회원가입
작별하지 않는다 :한강 장편소설
책
저자 한강
출판사 문학동네
출판연도 2021
ISBN 9788954682152
청구기호 813.7 2021-118
대출 가능 여부 대출가능
목록으로
대출하기
AITECH 도서관 © Spring 2026

조회(Select) - List

조회(Select) - List

- BookDao에 getAllBook 재사용

- BookController에 조회 쿼리 적용될 URL 연결

```
@RequestMapping("/managementbook")
public String managementbook(Model model) {
    List<BookVo> books = bookDao.getAllBook();
    model.addAttribute("books", books);
    return "admin/management_book";
}
```

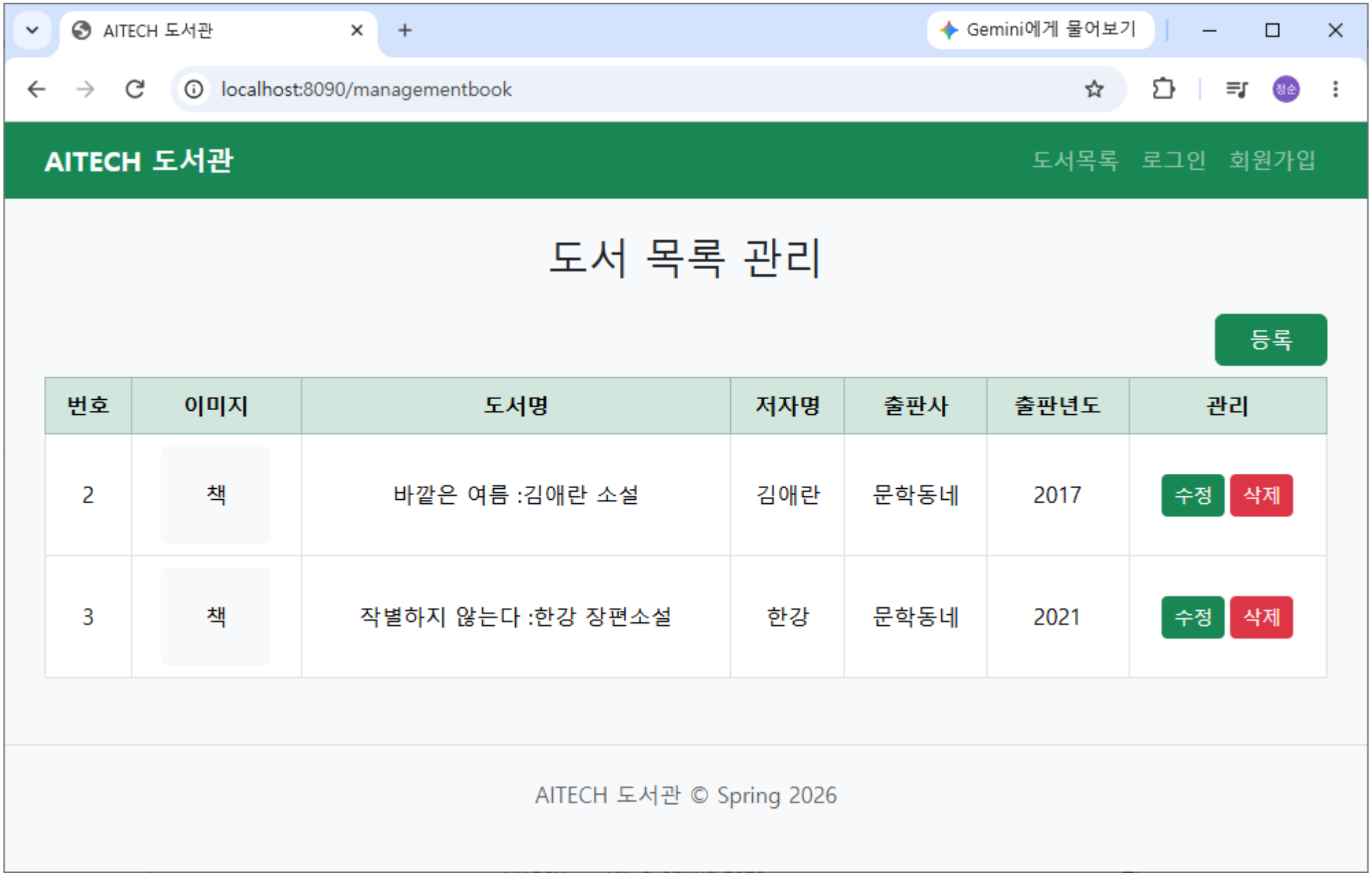
조회(Select) - List

management_book 내용 출력 수정

```
<c:forEach var="book" items="${books}">
    <tr>
        <td>${book.b_no}</td>
        <td><div class="bg-light rounded py-4 mx-auto" style="width: 80px;">책</div></td>
        <td>${book.b_name}</td>
        <td>${book.b_author}</td>
        <td>${book.b_publisher}</td>
        <td>${book.b_publish_year}</td>
        <td>
            <div class="d-flex justify-content-center gap-1">
                <a href="modifybook?b_no=${book.b_no}" class="btn btn-sm btn-success">수정</a>

            </div>
        </td>
    </tr>
</c:forEach>
```

도서관리 참고화면


AITECH 도서관
Gemini에게 물어보기
localhost:8090/managementbook
AITECH 도서관
도서목록 로그인 회원가입
도서 목록 관리
등록
번호 이미지 도서명 저자명 출판사 출판년도 관리
2 책 바깥은 여름 :김애란 소설 김애란 문학동네 2017 수정 삭제
3 책 작별하지 않는다 :한강 장편소설 한강 문학동네 2021 수정 삭제
AITECH 도서관 © Spring 2026

수정(Update)

수정(Update)

BookDao에 updateBook 추가

```
public void updateBook(BookVo bookVo) {
    String sql = "UPDATE book SET b_name = ?, b_author = ?, b_publisher = ?, b_publish_year = ?, " +
                 "b_isbn = ?, b_call_number = ?, b_rental_able = ?, b_reg_date = now(), b_mod_date = now() " +
                 "WHERE b_no = ?";
    jdbcTemplate.update(sql, bookVo.getB_name(), bookVo.getB_author(), bookVo.getB_publisher(),
            bookVo.getB_publish_year(), bookVo.getB_isbn(), bookVo.getB_call_number(),
            bookVo.getB_rental_able(), bookVo.getB_no());
}
```

BookController에 조회 쿼리 적용될 URL 연결

```
@RequestMapping("/modifybook")
public String modifybook(@RequestParam("b_no") int b_no, Model model) {
    model.addAttribute("book", bookDao.getDetailBook(b_no));
    return "admin/modify_book";
}

@RequestMapping("/modifybookconfirm")
public String modifybookconfirm(BookVo bookVo) {
    bookDao.updateBook(bookVo);
    return "redirect:managementbook";
}
```

수정(Update)

modify_book 내용 출력 수정

```
<form medhod="post" action="modifybookconfirm">
    <input type="hidden" name="b_no" value="${book.b_no}">

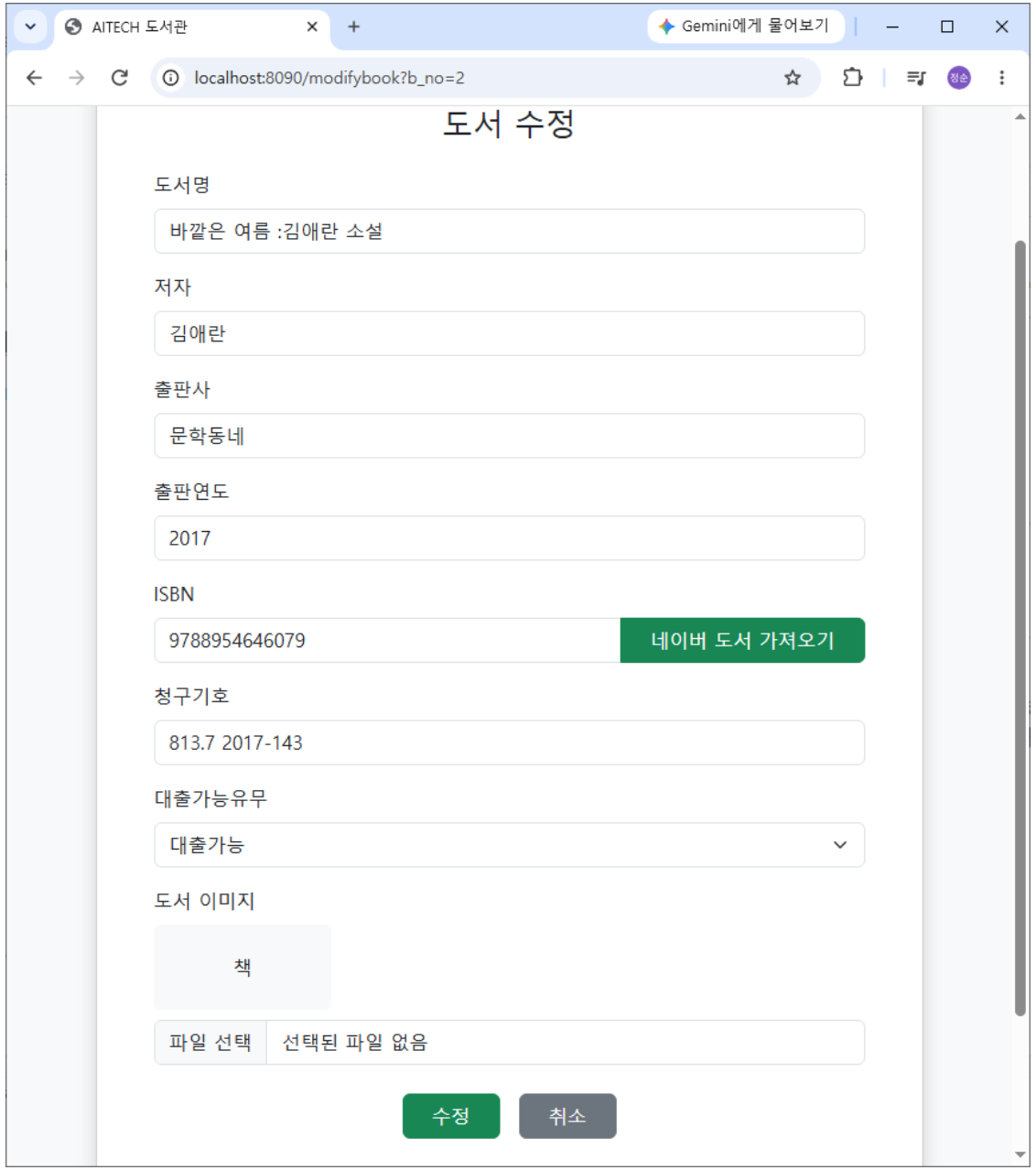
    <label class="form-label">도서명</label>
    <input type="text" name="b_name" class="form-control mb-3" value="${book.b_name}">

    <%-- 저자, 출판사, 출판년도, ISBN, 청구기호 동일하게 수정 --%>

    <label class="form-label">대출가능유무</label>
    <select class="form-select mb-3" name="b_rental_able">
        <option value="1" ${book.b_rental_able == 1 ? 'selected' : ''}>대출가능</option>
        <option value="0" ${book.b_rental_able == 0 ? 'selected' : ''}>대출불가</option>
    </select>

    <div class="d-flex justify-content-center gap-3">
        <button type="submit" class="btn btn-success px-4">수정</button>
        <a href="managementbook" class="btn btn-secondary px-4">취소</a>
    </div>
</form>
```

도서수정 참고화면


AITECH 도서관
Gemini에게 물어보기
localhost:8090/modifybook?b_no=2
도서 수정
도서명
바깥은 여름 :김애란 소설
저자
김애란
출판사
문학동네
출판연도
2017
ISBN
9788954646079
네이버 도서 가져오기
청구기호
813.7 2017-143
대출가능유무
대출가능
도서 이미지
책
파일 선택
선택된 파일 없음
수정
취소

삭제(Delete)

삭제(Delete)

■ BookDao에 deleteBook 추가

```
public void deleteBook(int b_no) {
    String sql = "DELETE FROM book WHERE b_no = ?";
    int rows = jdbcTemplate.update(sql, b_no);
}
```

■ BookController에 조회 쿼리 적용될 URL 연결

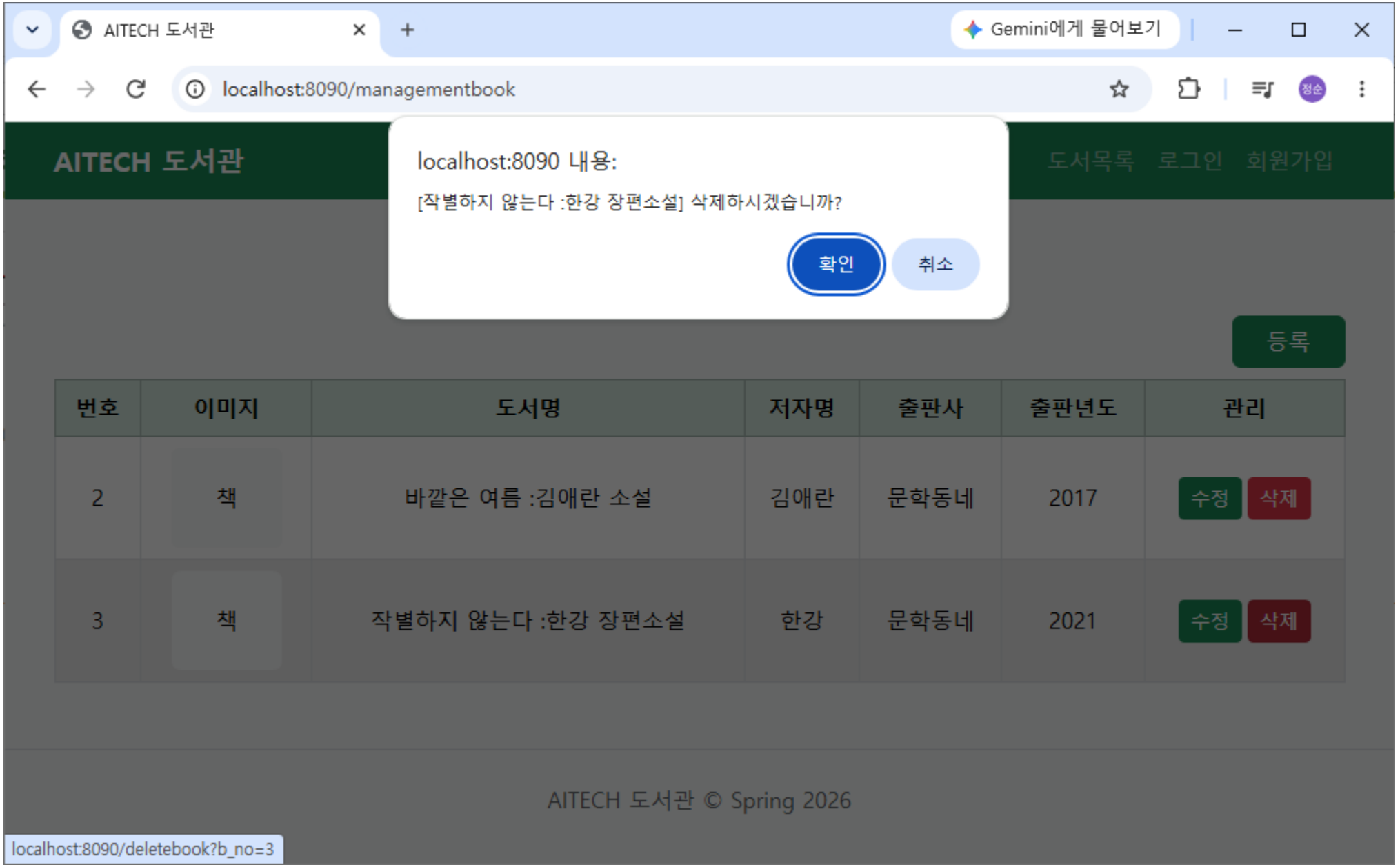
```
@RequestMapping("/deletebook")
public String deleteybook(@RequestParam("b_no") int b_no) {
    bookDao.deleteBook(b_no);
    return "redirect:managementbook";
}
```

수정(Update)

modify_book 내용 출력 수정

```
<div class="d-flex justify-content-center gap-1">
    <a href="modifybook?b_no=${book.b_no}" class="btn btn-sm btn-success">수정</a>
    <a href="deletebook?b_no=${book.b_no}" class="btn btn-sm btn-danger"
    onclick="return confirm('[${book.b_name}] 삭제하시겠습니까?');">삭제</a>
</div>
```

도서삭제 참고화면


AITECH 도서관
Gemini에게 물어보기
localhost:8090/managementbook
AITECH 도서관
도서목록 로그인 회원가입
localhost:8090 내용:
[작별하지 않는다 :한강 장편소설] 삭제하시겠습니까?
확인
취소
등록
번호 이미지 도서명 저자명 출판사 출판년도 관리
2 책 바깥은 여름 :김애란 소설 김애란 문학동네 2017 수정 삭제
3 책 작별하지 않는다 :한강 장편소설 한강 문학동네 2021 수정 삭제
AITECH 도서관 © Spring 2026
localhost:8090/deletebook?b_no=3

파일 업로드(File Upload)

파일 업로드(File Upload)

register_book 수정

- enctype="multipart/form-data" -> 내용이 추가되지 않으면 에러 없이 파일 누락됨

```
<form method="post" action="registerbookconfirm" enctype="multipart/form-data">

    <label class="form-label">도서 이미지</label>
    <input type="file" name="b_img_file" class="form-control mb-4">

    <div class="d-flex justify-content-center gap-3">
        <button type="submit" class="btn btn-success px-4">등록</button>
        <a href="managementbook" class="btn btn-secondary px-4">취소</a>
    </div>
</form>
```

파일 업로드(File Upload)

BookController 수정

- enctype="multipart/form-data" -> 내용이 추가되지 않으면 에러 없이 파일 누락됨

```
@RequestMapping("/registerbookconfirm")
public String registerbookconfirm(BookVo bookVo, @RequestParam("b_img_file") MultipartFile file) throws Exception {
    if(!file.isEmpty()) { //선택 파일이 있으면
        String uploadDir = System.getProperty("user.dir") + "/upload/book_images/"; //파일 저장 경로
        File dir = new File(uploadDir); //경로 주소 객체 설정
        if(!dir.exists()) //해당 경로 존재하지 않으면
            dir.mkdirs(); //해당 경로 디렉토리 생성

        //저장할 파일명 설정 오늘 날짜 형식(yyyyMMdd_HHmmss_) + 실제파일명.확장자
        String fileName=new SimpleDateFormat("yyyyMMdd_HHmmss_").format(new Date()) + file.getOriginalFilename();
        file.transferTo(new File(uploadDir + fileName));
        bookVo.setB_img(fileName); //실제파일명.확장자만 DB 저장
    }
    bookDao.insertBook(bookVo);
    return "admin/register_book";
}
```

파일 업로드(File Upload)

application.properties 추가

- spring의 설정 값 변경 -> 반드시 재시작 필요
- Localhost:8090/DB등록 이미지명.확장자로 확인 가능

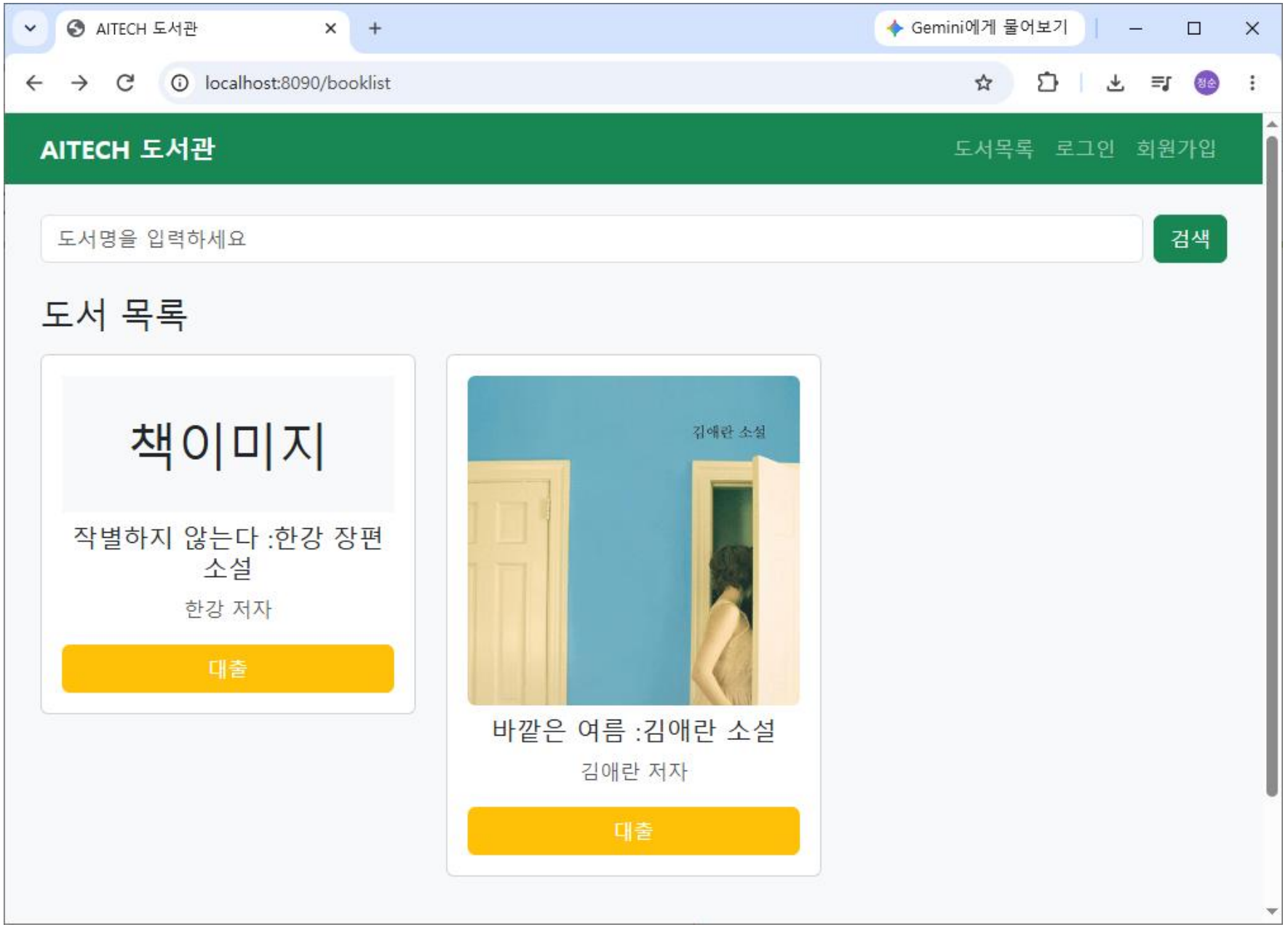
```
spring.web.resources.static-locations=classpath:/static/,file:${user.dir}/upload/book_images/
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB
```

book_list 수정

- 이미지 class 및 style 추가하여 조정

```
<c:choose>
    <c:when test="${not empty book.b_img}">
        <img src="${book.b_img}" class="w-100 rounded mb-2" style="height: 260px; object-fit: cover;">
    </c:when>
    <c:otherwise>
        <div class="bg-light py-4 mb-2" style="font-size:40px">책이미지</div>
    </c:otherwise>
</c:choose>
```

이미지 적용 후 도서목록 참고화면



■ book_detail 수정

```
<c:choose>
    <c:when test="${not empty book.b_img}">
        <img src="${book.b_img}" class="rounded mx-auto d-block"
         style="width: 180px; height: 240px; object-fit: cover;">
    </c:when>
    <c:otherwise>
        <div class="bg-light rounded py-4 mx-auto" style="width:80px">책이미지</div>
    </c:otherwise>
</c:choose>
```

management_book 수정

```
<c:choose>
    <c:when test="${not empty book.b_img}">
        <img src="${book.b_img}" class="rounded mx-auto d-block"
         style="width: 80px; height: 110px; object-fit: cover;">
    </c:when>
    <c:otherwise>
        <div class="bg-light rounded py-4 mx-auto" style="width:80px">책이미지</div>
    </c:otherwise>
</c:choose>
```

■ modify_book 수정

```
<c:choose>
    <c:when test="${not empty book.b_img}">
        <img src="${book.b_img}" class="rounded mx-auto d-block"
         style="width: 80px; height: 110px; object-fit: cover;">
    </c:when>
    <c:otherwise>
        <div class="bg-light rounded py-4 mx-auto" style="width:80px">책이미지</div>
    </c:otherwise>
</c:choose>
```

파일 업로드(File Upload)

BookController 수정

- 등록 및 수정시 파일 등록 반복으로 method로 분리

```
private String saveBookImg(MultipartFile file) throws Exception {
    String uploadDir = System.getProperty("user.dir") + "/upload/book_images/";
    File dir = new File(uploadDir);
    if(!dir.exists())
        dir.mkdirs();

    String fileName=new SimpleDateFormat("yyyyMMdd_HHmmss_").format(new Date()) + file.getOriginalFilename();
    file.transferTo(new File(uploadDir + fileName));
    return fileName;
}
```

BookController 수정

```
@RequestMapping("/registerbookconfirm")
public String registerbookconfirm(BookVo bookVo, @RequestParam("b_img_file") MultipartFile file) throws Exception {
    if(!file.isEmpty()) { //선택 파일이 있으면
        bookVo.setB_img(saveBookImg(file)); //실제파일명.확장자만 DB 저장
    }
    bookDao.insertBook(bookVo);
    return "admin/register_book";
}

@RequestMapping("/modifybookconfirm")
public String modifybookconfirm(BookVo bookVo, @RequestParam("b_img_file") MultipartFile file) throws Exception {
    BookVo oldBook = bookDao.getDetailBook(bookVo.getB_no());

    if(!file.isEmpty()) { //선택 파일이 있으면
        bookVo.setB_img(saveBookImg(file)); //실제파일명.확장자만 DB 저장

        if(oldBook.getB_img() != null && !oldBook.getB_img().isEmpty()) {
            new File(System.getProperty("user.dir")
                    + "/upload/book_images/" + oldBook.getB_img()).delete();
        }
    } else {
        bookVo.setB_img(oldBook.getB_img());
    }
    bookDao.updateBook(bookVo);
    return "redirect:managementbook";
}
```

기존이미지 삭제

파일 업로드(File Upload)

코드 최적화

- 사진 등록(saveBookImg) 및 삭제(deleteBookImg) 메서드화
- 저장 경로 상수 설정

```
private static final String UPLOAD_DIR =
        System.getProperty("user.dir") + "/upload/book_images/";

@RequestMapping("/deletebook")
public String deleteybook(@RequestParam("b_no") int b_no) {
    BookVo oldBook = bookDao.getDetailBook(b_no);
    bookDao.deleteBook(b_no);
    deleteBookImg(oldBook.getB_img());
    return "redirect:managementbook";
}

private void deleteBookImg(String b_img) {
    if(b_img != null && !b_img.isEmpty()) {
        new File(UPLOAD_DIR + b_img).delete();
    }
}
```

파일 업로드(File Upload)

■ 코드 최적화

```
@RequestMapping("/registerbookconfirm")
public String registerbookconfirm(BookVo bookVo, @RequestParam("b_img_file") MultipartFile file) throws Exception {
    if(!file.isEmpty()) { //선택 파일이 있으면
        bookVo.setB_img(saveBookImg(file)); //실제파일명.확장자만 DB 저장
    }
    bookDao.insertBook(bookVo);
    return "admin/register_book";
}

@RequestMapping("/modifybookconfirm")
public String modifybookconfirm(BookVo bookVo, @RequestParam("b_img_file") MultipartFile file) throws Exception {
    BookVo oldBook = bookDao.getDetailBook(bookVo.getB_no());

    if(!file.isEmpty()) { //선택 파일이 있으면
        bookVo.setB_img(saveBookImg(file)); //실제파일명.확장자만 DB 저장
        deleteBookImg(oldBook.getB_img());
    } else {
        bookVo.setB_img(oldBook.getB_img());
    }
    bookDao.updateBook(bookVo);
    return "redirect:managementbook";
}
```

파일 업로드(File Upload)

■ 코드 최적화

```
private String saveBookImg(MultipartFile file) throws Exception {
    File dir = new File(UPLOAD_DIR);
    if(!dir.exists())
        dir.mkdirs();

    String fileName=new SimpleDateFormat("yyyyMMdd_HHmmss_").format(new Date()) + file.getOriginalFilename();
    file.transferTo(new File(UPLOAD_DIR + fileName));
    return fileName;
}
```

이미지 적용 후 도서관리 참고화면

AITECH 도서관

Gemini에게 물어보기

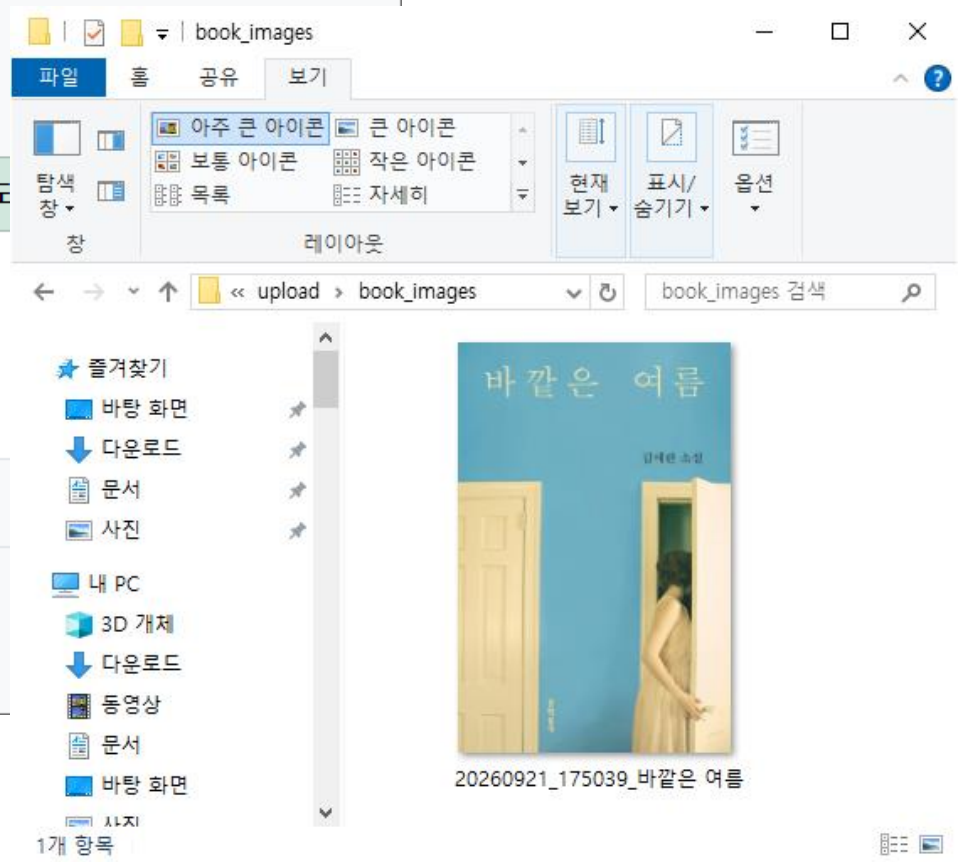
localhost:8090/managementbook

AITECH 도서관 도서목록 로그인 회원가입

도서 목록 관리

번호	이미지	도서명	저자명	출판사	출판년도
4		바깥은 여름 :김애란 소설	김애란	문학동네	2017

AITECH 도서관 © Spring 2026



Member

Member

MemebeVo

- member package 생성
- MemberVo Class 생성

```
public class MemberVo {
    private int m_no;
    private String m_id;
    private String m_pw;
    private String m_name;
    private String m_gender;
    private String m_phone;
    private String m_mail;
    private String m_role;
    private String m_reg_date;
    private String m_mod_date;
```

MemberRowMapper

- MemberRowMapper 생성

```
public class MemberRowMapper implements RowMapper<MemberVo> {

    @Override
    public MemberVo mapRow(ResultSet rs, int rowNum) throws SQLException {
        MemberVo member = new MemberVo();
        member.setM_no(rs.getInt("m_no"));
        member.setM_id(rs.getString("m_id"));
        member.setM_pw(rs.getString("m_pw"));
        member.setM_name(rs.getString("m_name"));
        member.setM_gender(rs.getString("m_gender"));
        member.setM_phone(rs.getString("m_phone"));
        member.setM_mail(rs.getString("m_mail"));
        member.setM_role(rs.getString("m_role"));
        return member;
    }
}
```

MemberDao

- MemberDao 생성 후 @Repository 어노테이션 추가

```
@Repository
public class MemberDao {
    private JdbcTemplate jdbcTemplate;

    @Autowired
    public MemberDao(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public void insertMember(MemberVo memberVo) {
        String sql = "INSERT INTO member(m_id, m_pw, m_name, m_gender,"
                + " m_phone, m_mail, m_role, m_reg_date, m_mod_date)"
                + " VALUES(?,?,?,?,?,?,'USER',now(),now())";
        jdbcTemplate.update(sql, memberVo.getM_id(), memberVo.getM_pw(),
                memberVo.getM_name(), memberVo.getM_gender(), memberVo.getM_phone(), memberVo.getM_mail());
    }

    public MemberVo getDetailMember(int m_no) {
        String sql = "SELECT * FROM member WHERE m_no = ?";
        return jdbcTemplate.queryForObject(sql, new MemberRowMapper(), m_no);
    }
```

MemberDao

- MemberDao 생성 후 @Repository 어노테이션 추가

```
    public void updateMember(MemberVo memberVo) {
        String sql = "UPDATE member SET m_pw=?, m_name=?, m_gender=?, m_phone=?,"
                + " m_mail=?, m_mod_date=now() WHERE m_no=?";
        jdbcTemplate.update(sql, memberVo.getM_pw(), memberVo.getM_name(),
                memberVo.getM_gender(), memberVo.getM_phone(), memberVo.getM_mail(), memberVo.getM_no());
    }
}
```

MemberController

- MemberController 생성 후 @Controller 어노테이션 추가

```
@Controller
public class MemberController {
    @Autowired
    private MemberDao memberDao;

    @RequestMapping("/registermember")
    public String registerMember() {
        return "user/register_member";
    }

    @RequestMapping("/registermemberconfirm")
    public String registerMemberConfirm(MemberVo memberVo) {
        memberDao.insertMember(memberVo);
        return "redirect:/loginmember";    // 가입 후 로그인 화면(아직 골격)으로
    }

    @RequestMapping("/modifymember")
    public String modifyMember(@RequestParam("m_no") int m_no, Model model) {
        model.addAttribute("member", memberDao.getDetailMember(m_no));
        return "user/modify_member";
    }
```

MemberController

- MemberController 생성 후 @Controller 어노테이션 추가

```
    @RequestMapping("/modifymemberconfirm")
    public String modifyMemberConfirm(MemberVo memberVo) {
        if (memberVo.getM_pw() == null || memberVo.getM_pw().isEmpty()) {
            MemberVo oldMember = memberDao.getDetailMember(memberVo.getM_no());
            memberVo.setM_pw(oldMember.getM_pw());
        }
        memberDao.updateMember(memberVo);
        return "redirect:/booklist";
    }
}
```

Thank you!